

Pre-entrega TPE · Tema 9: Protección de Servicios con WAF para *The Store*

Redes de Información · ITBA · 1C 2026 · Grupo 2

Mauro Vella — [Legajo] · Enrique Castillo — 68321 · Federico Inti García Lauberer — 61374

1. Problemática y contexto

The Store es un sitio de e-commerce compuesto por cinco microservicios (una interfaz web y cuatro backends: catálogo, carritos, pedidos y checkout) que corren sobre un cluster Kubernetes local (Kind) con **ingress-nginx** como único punto de entrada desde Internet. Por diseño, los backends no deberían ser accesibles desde afuera; hoy **lo son**.

Una auditoría hecha por el grupo contra `http://localhost` el 16 de abril encontró **10 vectores de ataque explotables sin usuario ni contraseña**, en menos de tres minutos con sólo un navegador. Los más graves para el negocio son:

- **Los servicios “internos” se pueden alcanzar desde afuera.** El componente que orquesta la UI tiene un proxy (`/proxy/...`) que reenvía requests a los backends. Eso hace que carritos, pedidos y checkout —pensados como servicios *ClusterIP* (solo visibles dentro del cluster)— respondan desde Internet. **Concretamente:** un atacante puede pedir `/proxy/carts/admin` y obtener el carrito del usuario `admin` sin autenticarse. Esto se llama **IDOR** (*Insecure Direct Object Reference*, lectura no autorizada usando identificadores adivinables) y en 2022 Optus (Australia) perdió datos de 10 millones de clientes por un error idéntico.
- **Combinando el proxy con una URL maliciosa, se escapa del propio backend.** Pedir `/proxy/catalog/../../../../actuator/prometheus` —una técnica conocida como *path traversal*, donde `../` fuerza a subir niveles de directorio— devuelve **25 KB** del panel de diagnóstico interno. En esos 25 KB hay: nombres de los cuatro microservicios, capacidad y uso de disco, versión exacta del framework, qué URLs están siendo llamadas en vivo por otros clientes y qué excepciones está tirando el sistema. **Para el negocio:** es como dejar colgado, en la puerta del local, el plano del sistema y el registro de qué está haciendo cada cliente en tiempo real. Todo atacante que quiera dañar a *The Store* empieza leyendo eso.
- **Cualquier atacante se puede hacer pasar por cualquier IP.** El sitio confía en headers como `X-Forwarded-For` sin verificar de dónde vienen. En los logs queda registrado que las requests vienen de IPs falsas, lo que **anula los bloqueos por IP** y complica cualquier investigación forense posterior a un incidente.
- **No hay límite de velocidad.** En una prueba controlada, **100 requests seguidas tardaron 597 ms (167 por segundo)** y todas recibieron respuesta 200 OK. Eso permite: (i) escapar el catálogo completo en minutos, (ii) hacer fuerza bruta contra cupones o tokens de sesión, y (iii) saturar la app con un único atacante. **Para el negocio:** pérdida de ventas durante horas pico y ventaja competitiva para quien copie el catálogo.
- **Ningún antivirus de red detecta los escaneos conocidos.** User-Agents de herramientas automatizadas como `sqlmap` o `nikto` pasan sin alarma y devuelven 200 OK. **Esto significa que un atacante puede mapear todos los puntos débiles del sitio sin ser detectado**, y el equipo operativo se entera recién cuando ya hubo daño.
- **Faltan los seis headers de seguridad estándar** (*HSTS*, *Content-Security-Policy*, *X-Frame-Options* y tres más). Ausencia de *X-Frame-Options* permite *clickjacking*: montar un sitio visualmente idéntico que enmarca el nuestro y captura el clic “Comprar” del cliente. Ausencia de *HSTS* permite interceptar la sesión si el cliente entró desde una WiFi pública.

La buena noticia: **todas las entradas de Internet pasan por un único punto** (el `ingress-nginx`). Eso nos permite resolver los seis hallazgos de arriba insertando **una sola pieza de software en ese punto**, sin tocar el código de ninguno de los cinco microservicios.

2. Diseño de la solución

Proponemos desplegar **ModSecurity v3** (motor de inspección de tráfico HTTP que puede registrar o bloquear requests según reglas) junto con el **OWASP Core Rule Set** (conjunto de alrededor de 2000 reglas mantenidas por la fundación OWASP que cubren los ataques web más comunes) dentro del `ingress-nginx-controller` que ya existe en el cluster.

Cómo funciona en la práctica: cada request entrante pasa primero por ModSecurity. El motor parsea

URL, headers y body, acumula un puntaje de sospecha según cuántas reglas matchean y, si supera el umbral, devuelve **403 Forbidden** en lugar de reenviar al backend. Los cinco microservicios no se enteran del cambio.

Etapas de despliegue:

- 1. **DetectionOnly** (modo “solo registrar”): el WAF anota en un log qué *habría* bloqueado. Sirve para descubrir y corregir falsos positivos sobre tráfico legítimo antes de activar el bloqueo real.
- 2. **On** (modo “bloquear”): una vez tuneado, el WAF pasa a responder 403 sobre las requests maliciosas.

Qué vamos a agregar encima del CRS base (porque el CRS no conoce la lógica específica de *The Store*):

- Reglas custom para **bloquear el panel** `/actuator/*` salvo `/actuator/health` (que Kubernetes usa para health-checks internos).
- Reglas custom para **cerrar el agujero del proxy**: denegar cualquier request a `/proxy/*` que contenga `..` en el path o que no traiga un header de sesión válido.
- Anotaciones en el Ingress para **aplicar límite de velocidad** (por ejemplo 10 requests/segundo por IP) y **agregar los seis headers de seguridad** que hoy faltan.

Ventaja principal del enfoque: el alcance del cambio queda acotado a archivos de configuración (ConfigMap del Ingress + anotaciones). No se modifica el código Java, Go ni Node de los cinco servicios, y cualquier ajuste futuro se hace en un solo lugar.

3. Scope del POC y casos de uso

Alcance comprometido para la entrega final

- Bloqueo efectivo de los endpoints de diagnóstico sensibles: `/actuator/info`, `/actuator/metrics` y `/actuator/prometheus` deben pasar de 200 OK a 403 Forbidden.
- Bloqueo de ataques de *path traversal* y SSRF (*Server-Side Request Forgery*, engañar al servidor para que él haga requests a destinos no previstos) sobre el proxy: por ejemplo `/proxy/catalog/../../../../actuator/prometheus` queda cortado.
- Detección y bloqueo de User-Agents de herramientas automáticas conocidas (sqlmap, nikto, nuclei).
- Límite de velocidad por IP aplicado desde el Ingress.
- Incorporación de los seis headers de seguridad HTTP en toda respuesta de la aplicación.
- Demostración en vivo de al menos cinco de los seis hallazgos, con captura de logs del WAF mostrando qué regla disparó cada bloqueo.

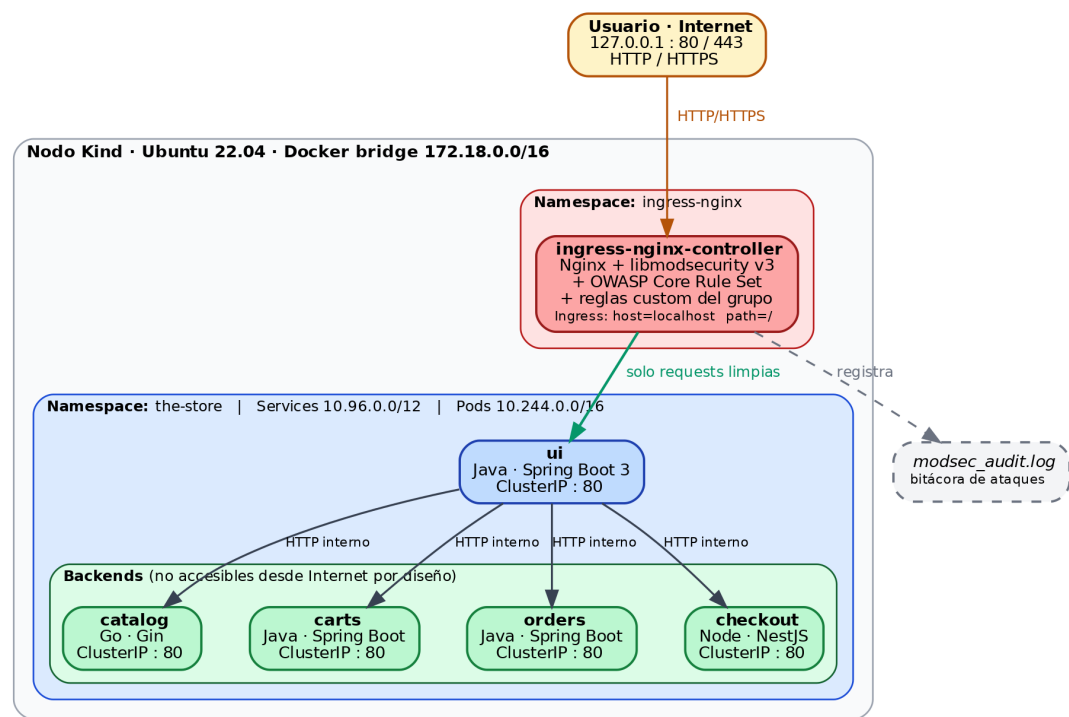
Casos de uso para la demo (patrón “antes / después”)

#	Caso	Pre-WAF	Post-WAF esperado
1	Exfiltración del panel <code>/actuator/prometheus</code>	200 OK, 25 KB	403 Forbidden
2	Path traversal <code>/proxy/catalog/../../../../actuator/info</code>	200 OK	403 Forbidden
3	Request con User-Agent: Nikto/2.5	200 OK	403 Forbidden (logged)
4	Burst de 100 requests en paralelo a /	100/100 responden	throttle activo
5	Headers de la home	0/6 presentes	6/6 presentes

Qué NO cubre la pre-entrega (para acotar expectativas)

- No reemplaza autenticación entre microservicios ni parches al código de los backends; es una capa adicional, no un sustituto.
- No aborda ataques de lógica de negocio (por ejemplo, abusar de un cupón de descuento aplicándolo cien veces): ese tipo de problemas requiere reglas específicas que no forman parte del alcance.
- No incluye tráfico HTTPS terminado afuera del cluster; el escenario del POC asume que el tráfico termina en el Ingress.

4. Diagrama de arquitectura



Bloque	CIDR	Rol
Host loopback	127.0.0.1/32	Usuario accede a <i>The Store</i> por 80 / 443
Docker bridge del nodo Kind	172.18.0.0/16	Red del contenedor del nodo
Pods del cluster	10.244.0.0/16	Comunicación entre pods
Services ClusterIP	10.96.0.0/12	Direcciones internas de los servicios

- **Sistema operativo de los nodos:** kindest/node basado en Ubuntu 22.04 LTS.
- **Protocolos:** HTTP/1.1 tanto entre usuario e Ingress como entre UI y backends; DNS interno de Kubernetes para la resolución de nombres.
- **Punto de inserción del WAF:** pod ingress-nginx-controller en el namespace ingress-nginx. ModSecurity v3 se activa como módulo del Nginx ya presente.
- **Cambios a aplicar:** un ConfigMap que activa ModSecurity y OWASP CRS, un archivo con reglas custom del grupo, y anotaciones en el Ingress para rate limit y headers.

5. Alternativas consideradas

Shadow Daemon (listado en el enunciado). Descartado: su modelo requiere instalar un *conector* dentro de cada aplicación protegida, y sólo provee conectores para PHP, Python y Perl. Como *The Store* está escrito en Java, Go y Node.js, el modelo no encaja. Además, su comunidad y base de reglas son notoriamente más chicas que las de OWASP CRS.

OpenWAF (listado en el enunciado). Descartado: el proyecto no recibe commits desde 2018 y su documentación está parcialmente en chino sin traducir. Elegir una herramienta sin mantenimiento activo es un riesgo operativo que no queremos asumir para una pieza de seguridad crítica.

WAF gestionado en la nube (Cloudflare, AWS WAF, Azure WAF). Descartado: la consigna pide implementar un componente propio y desplegarlo en el cluster local, por lo que un servicio externo con costo mensual no aplica al alcance.

Coraza (OWASP). Sucesor moderno de ModSecurity escrito en Go y compatible con las mismas reglas CRS. Técnicamente superior, pero no figura en el enunciado; queda mencionado como evolución futura.